

Universidad Nacional de Entre Ríos

Facultad de Ciencias y Tecnología — Sede Concepción del Uruguay

Licenciatura en Sistemas de Información

TRABAJO PRÁCTICO N.º 3

TODO API con Persistencia en PostgreSQL usando Docker

Informe de resolución y evidencia de pruebas

Cátedra	Programación Avanzada
Unidad temática	Persistencia de datos y contenedores
Alumno	Julián Olivera
Docente	Mg. Lic. Ernesto Ledesma
Entorno	Node.js 20 · Docker Compose · PostgreSQL 16

1. Objetivo del trabajo

El objetivo de esta práctica fue migrar la TODO API desarrollada en la Práctica 01, que almacenaba las tareas en un arreglo en memoria, hacia una arquitectura con persistencia real en PostgreSQL. Tanto la API como la base de datos se ejecutan como contenedores Docker independientes, orquestados con Docker Compose y comunicados a través de la red interna que Compose crea automáticamente.

El trabajo consistió concretamente en implementar los endpoints `PUT /tasks/:id` (RF-04) y `DELETE /tasks/:id` (RF-05), verificar el comportamiento de los cinco endpoints de la API incluyendo los casos de error, y comprobar empíricamente que los datos sobreviven al reinicio del contenedor de la API.

2. Arquitectura de la solución

La solución se compone de dos servicios definidos en `docker-compose.yml`:

Servicio	Imagen	Puerto	Rol
api	Build local (Node 20 Alpine)	3001	Servidor Express que expone la API REST
db	postgres:16-alpine	5432	Base de datos, con volumen <code>todo_pgdata</code>

Resolución de nombres: dentro de la red de Compose la API se conecta a la base usando `DB_HOST=db`, es decir el nombre del servicio, y nunca `localhost`. Dentro de un contenedor `localhost` apunta al propio contenedor, por lo que usarlo provocaría un error de conexión. Desde el host, en cambio, la base sí es accesible como `localhost:5432` gracias al mapeo de puertos.

El orden de arranque se resuelve con un `healthcheck` sobre el servicio `db` (usando `pg_isready`) combinado con `depends_on: condition: service_healthy` en el servicio `api`. De esta forma la API no intenta conectarse antes de que PostgreSQL esté listo para aceptar conexiones.

2.1 Modelo de datos

La tabla `tasks` se crea mediante el script `db/init.sql`, que PostgreSQL ejecuta automáticamente una única vez, al inicializar el volumen por primera vez:

```
CREATE TABLE IF NOT EXISTS tasks (  
  id          SERIAL PRIMARY KEY,  
  title       VARCHAR(255) NOT NULL,  
  description TEXT DEFAULT '',  
  status      VARCHAR(50) NOT NULL DEFAULT 'pending',  
  due_date    DATE,  
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),  
  updated_at  TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

3. Implementación de los endpoints

3.1 PUT /tasks/:id — RF-04

La consigna exigía que la actualización conserve los valores existentes de los campos que no vengan en el body. Esto se resolvió con `COALESCE`: la función devuelve el primer argumento no nulo, de modo que si un parámetro llega como `undefined` (traducido a `NULL` por el driver `pg`), se conserva el valor que ya tenía la columna.

```
app.put('/tasks/:id', async (req, res) => {
  try {
    const { id } = req.params
    const { title, description, status, dueDate } = req.body

    if (status && !VALID_STATUSES.includes(status)) {
      return res.status(400).json({ error: `Status must be one of: ${VALID_STATUSES.join(', ')}` })
    }

    const result = await pool.query(
      `UPDATE tasks
      SET title = COALESCE($1, title),
          description = COALESCE($2, description),
          status = COALESCE($3, status),
          due_date = COALESCE($4, due_date),
          updated_at = NOW()
      WHERE id = $5
      RETURNING *`,
      [title, description, status, dueDate, id]
    )

    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Task not found' })
    }

    res.json(result.rows[0])
  } catch (err) {
    res.status(500).json({ error: err.message })
  }
})
```

La cláusula `RETURNING *` cumple una doble función: devuelve la fila ya actualizada para responderla al cliente, y permite detectar si el `id` existía. Si no se afectó ninguna fila, `result.rows` llega vacío y se responde 404. El campo `updated_at` se refresca con `NOW()` en cada actualización.

3.2 DELETE /tasks/:id — RF-05

```
app.delete('/tasks/:id', async (req, res) => {
  try {
    const { id } = req.params
    const result = await pool.query('DELETE FROM tasks WHERE id = $1 RETURNING *', [id])
```

```
    if (result.rows.length === 0) {  
        return res.status(404).json({ error: 'Task not found' })  
    }  
  
    res.status(204).send()  
  } catch (err) {  
    res.status(500).json({ error: err.message })  
  }  
})
```

Se aplica el mismo patrón: `RETURNING *` permite distinguir entre una eliminación efectiva y un `id` inexistente. Al borrar correctamente se responde `204 No Content`, sin cuerpo en la respuesta.

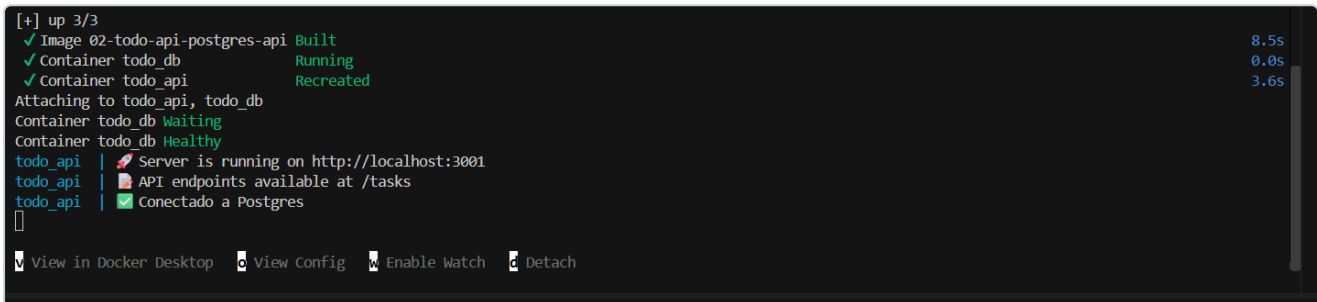
3.3 Prevención de inyección SQL

Ninguna consulta concatena valores provenientes del usuario dentro del string SQL. Todas utilizan parámetros posicionales (`$1`, `$2`, ...) y pasan los valores en un arreglo aparte. El driver envía la sentencia y los datos por separado, de modo que PostgreSQL siempre los interpreta como valores literales y nunca como código ejecutable, aun cuando contengan comillas o punto y coma.

4. Evidencia de ejecución

4.1 Levantado del entorno

El comando `docker compose up --build` construye la imagen de la API, espera a que el `healthcheck` de la base reporte estado `healthy` y recién entonces inicia el contenedor de la API, que confirma la conexión a PostgreSQL.

A terminal window showing the output of the command 'docker compose up --build'. The output indicates that the image '02-todo-api-postgres-api' was built (8.5s), the 'todo_db' container is running (0.0s), and the 'todo_api' container is recreated (3.6s). It then shows the API server running on http://localhost:3001, API endpoints available at /tasks, and a successful connection to PostgreSQL. At the bottom, there are links to 'View in Docker Desktop', 'View Config', 'Enable Watch', and 'Detach'.

```
[+] up 3/3
✓ Image 02-todo-api-postgres-api Built 8.5s
✓ Container todo_db Running 0.0s
✓ Container todo_api Recreated 3.6s
Attaching to todo_api, todo_db
Container todo_db waiting
Container todo_db Healthy
todo_api | 🚀 Server is running on http://localhost:3001
todo_api | 📄 API endpoints available at /tasks
todo_api | ✅ Conectado a Postgres
[]

View in Docker Desktop View Config Enable Watch Detach
```

Figura 1 — `docker compose up --build` : ambos contenedores levantados sin errores y conexión a Postgres establecida.

4.2 POST /tasks — creación de una tarea (201)

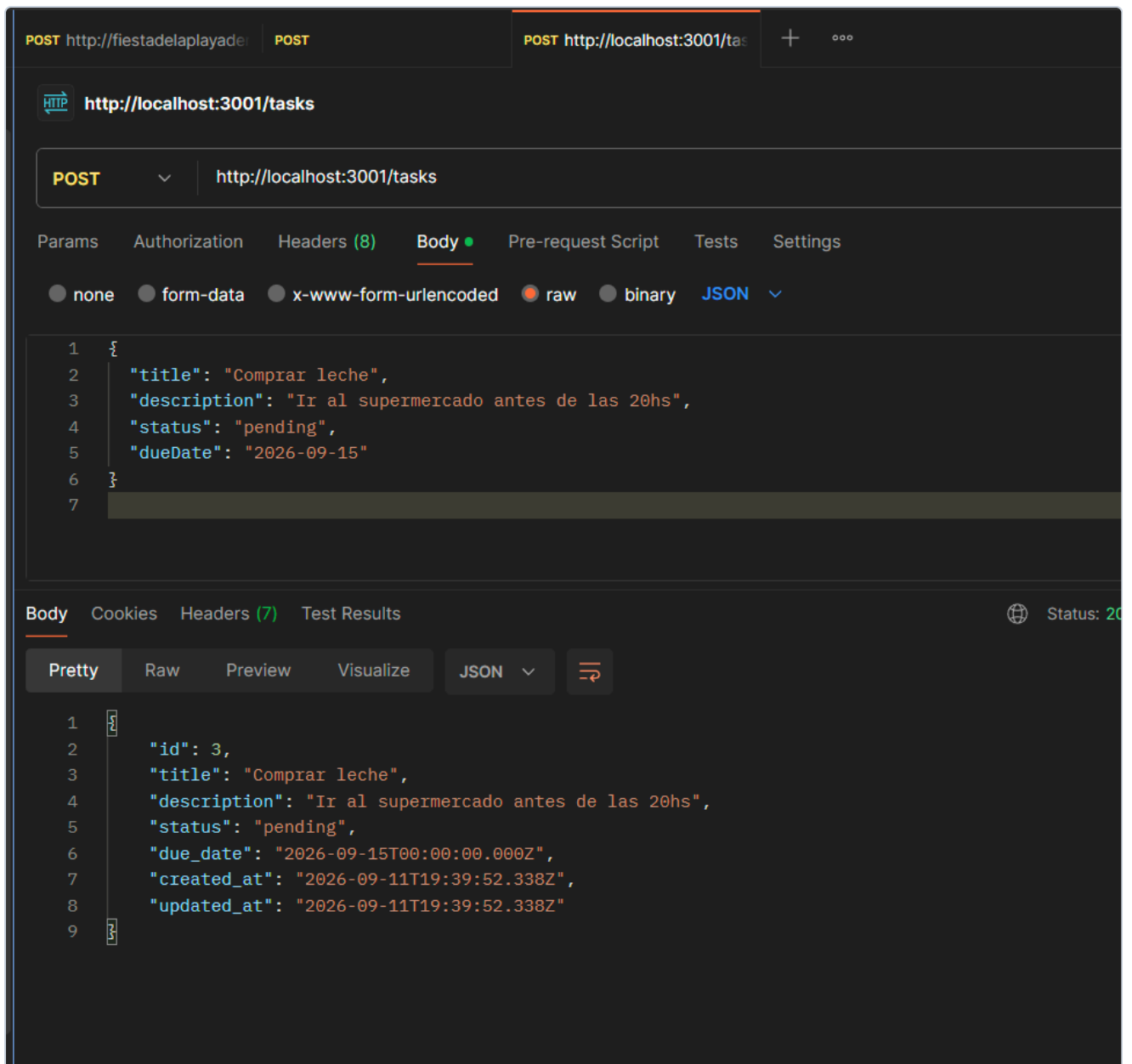


Figura 2 — `POST /tasks` responde 201 con el registro creado, incluyendo el `id` asignado por la secuencia y las marcas de tiempo.

4.3 GET /tasks — listado completo (200)

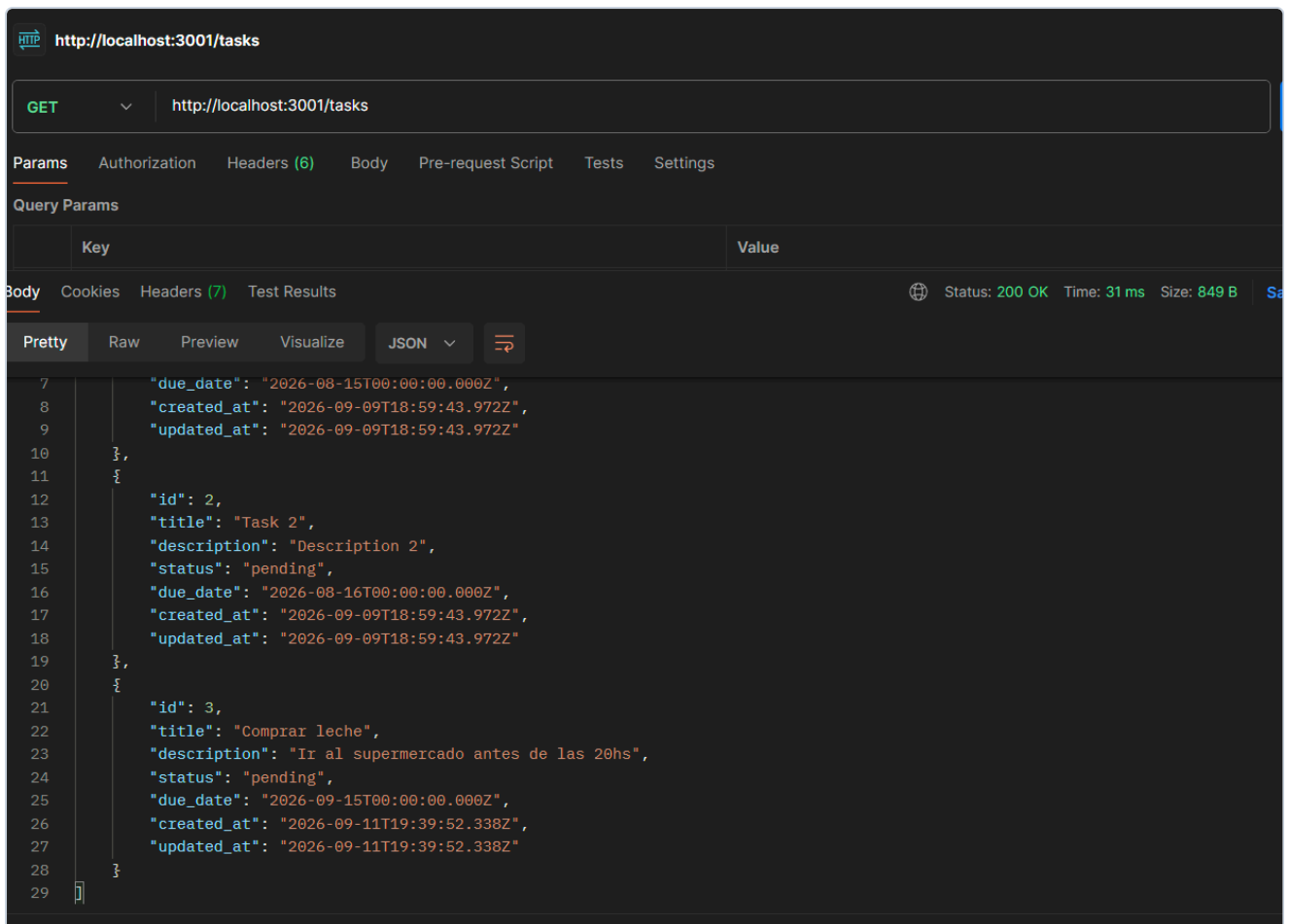


Figura 3 — `GET /tasks` devuelve las tareas ordenadas por `id`: las dos precargadas por `init.sql` más la recién creada.

4.4 GET /tasks/:id — consulta individual (200)

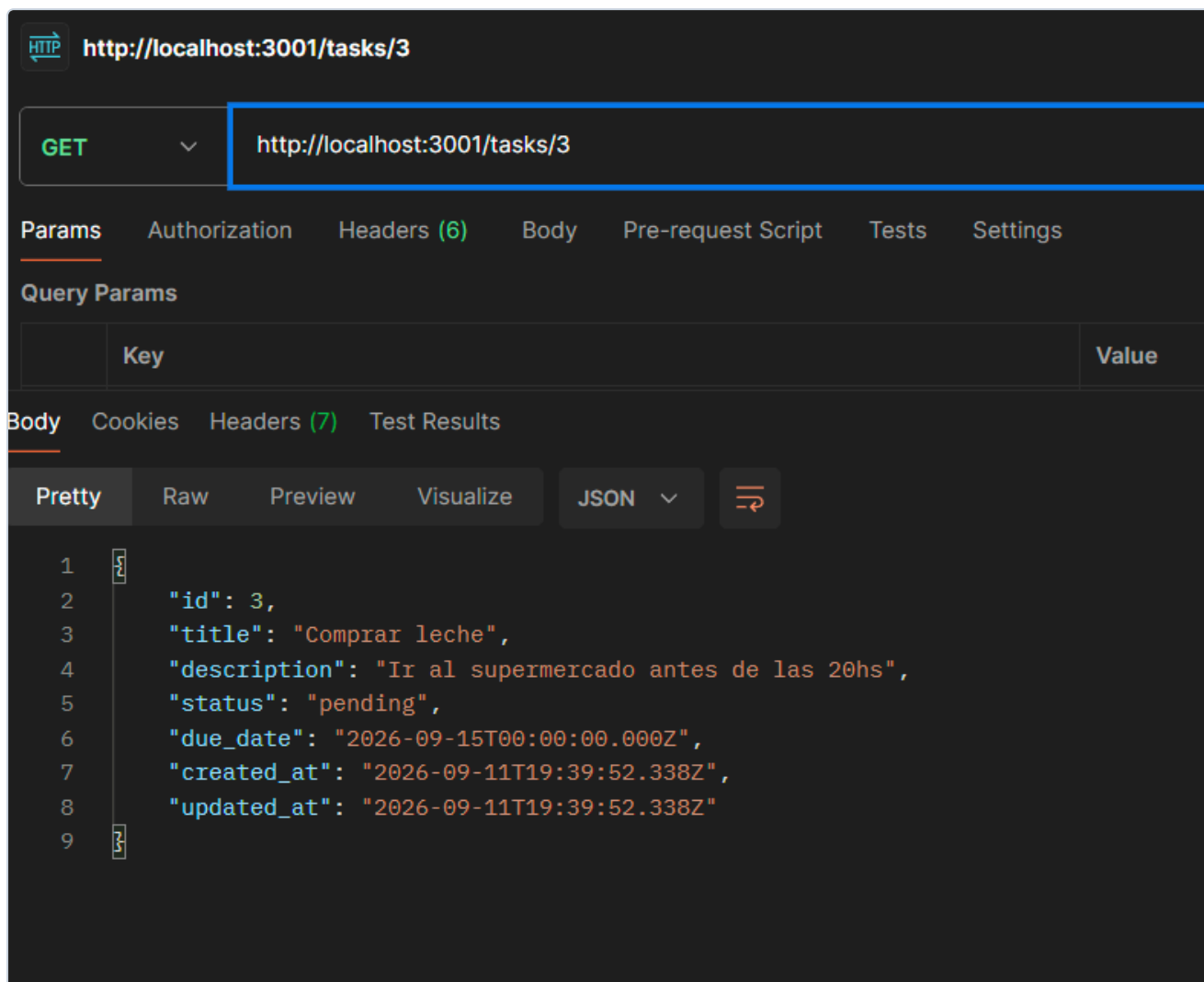


Figura 4 — `GET /tasks/3` devuelve únicamente la tarea solicitada.

4.5 PUT /tasks/:id — actualización (200)

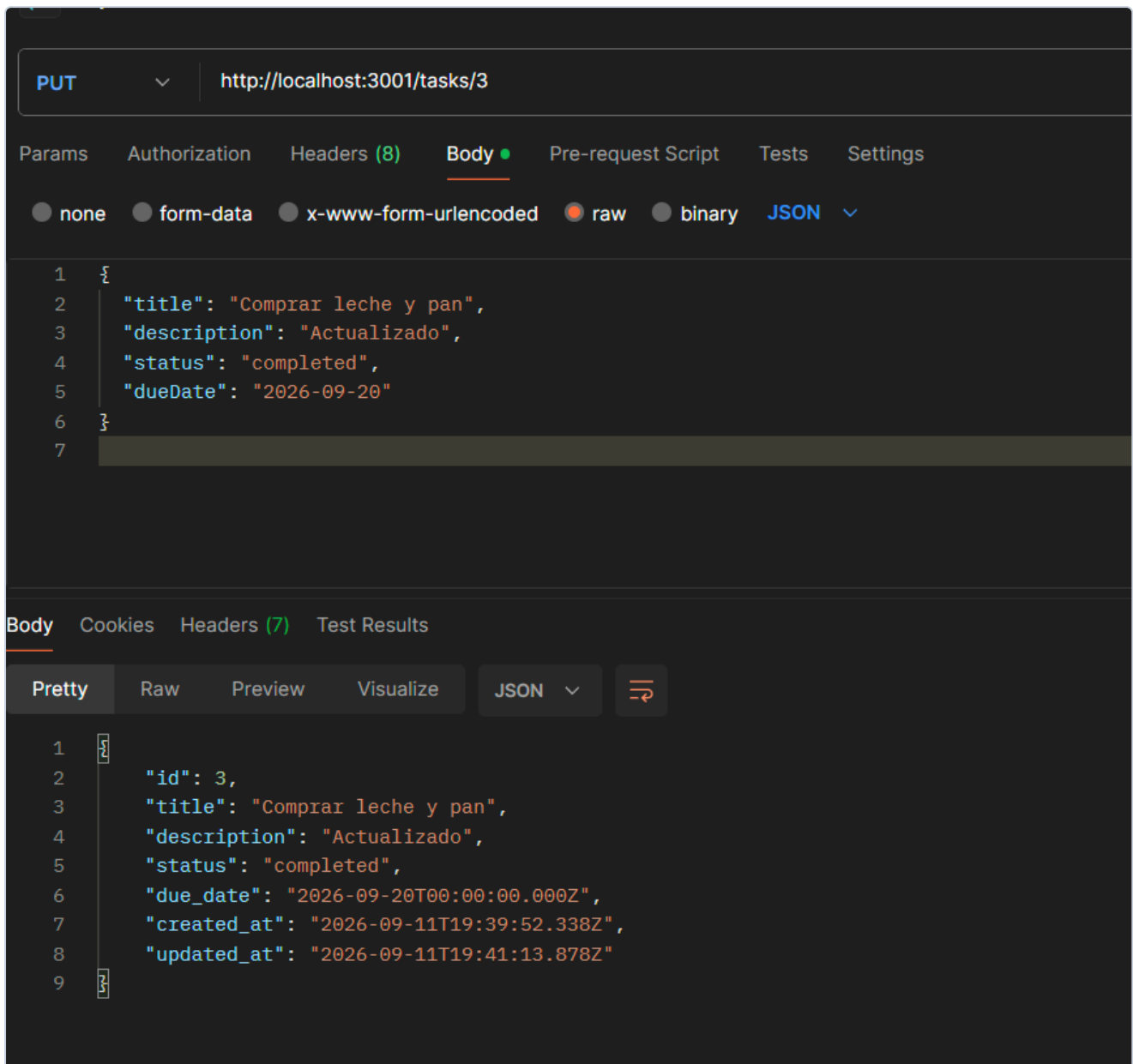


Figura 5 — `PUT /tasks/3` : los campos enviados se actualizan y `updated_at` pasa de 19:39:52 a 19:41:13, mientras `created_at` permanece sin cambios.

Verificación del criterio de aceptación: la comparación entre `created_at` (19:39:52.338Z) y `updated_at` (19:41:13.878Z) en la respuesta demuestra que `NOW()` se aplicó correctamente durante el UPDATE.

4.6 DELETE /tasks/:id — eliminación (204)

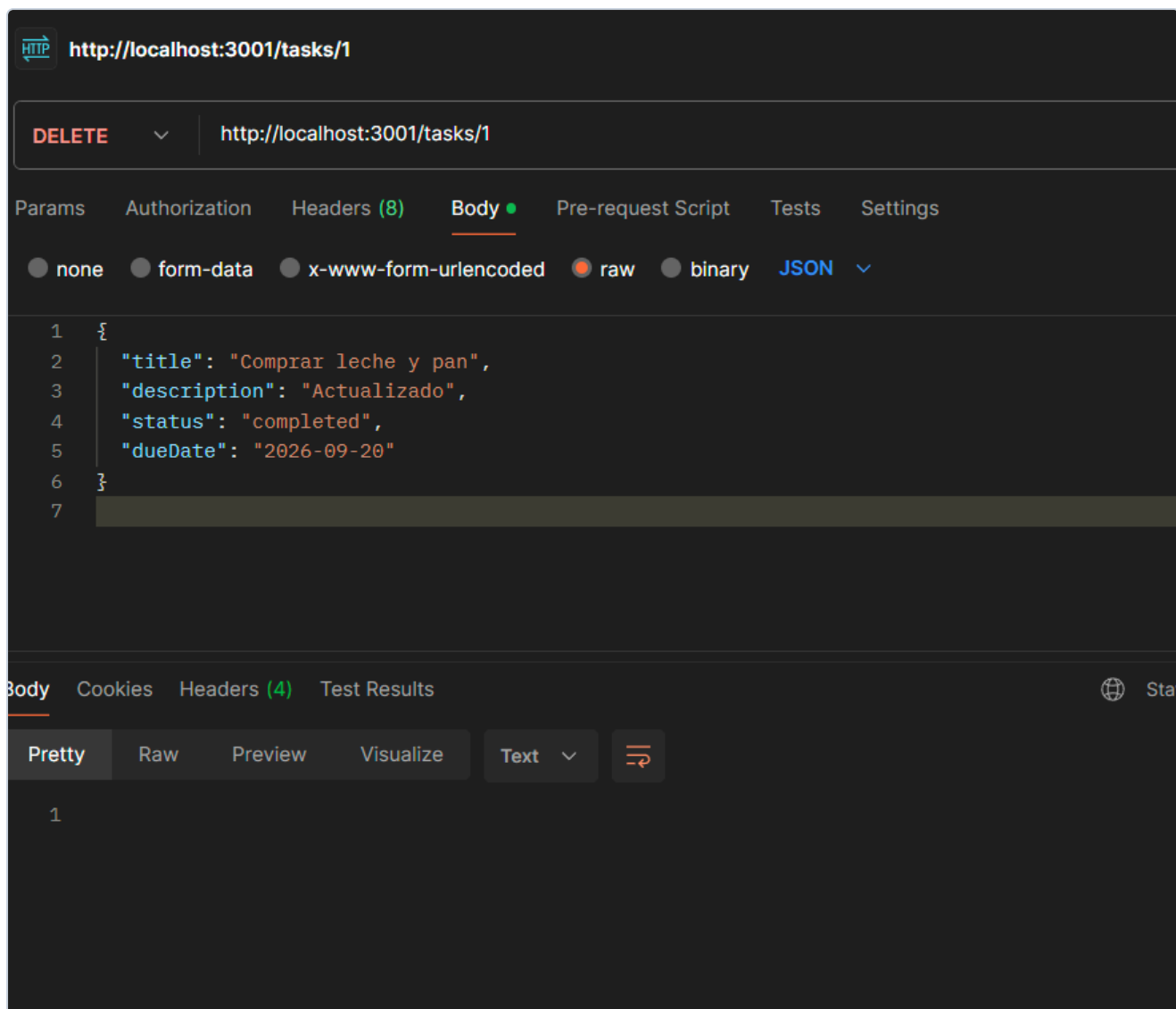


Figura 6 — `DELETE /tasks/1` responde sin contenido en el cuerpo, tal como corresponde a un 204 No Content.

4.7 Casos de error: 404 y 400

Se verificaron los tres endpoints que operan sobre un `id` puntual utilizando un identificador inexistente, junto con las validaciones de entrada del POST:

```
$ curl -i http://localhost:3001/tasks/9999
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8

{"error":"Task not found"}

$ curl -i -X PUT http://localhost:3001/tasks/9999 -H "Content-Type: application/json" -d
'{"status":"completed"}'
HTTP/1.1 404 Not Found

{"error":"Task not found"}

$ curl -i -X DELETE http://localhost:3001/tasks/9999
HTTP/1.1 404 Not Found

{"error":"Task not found"}

$ curl -i -X POST http://localhost:3001/tasks -H "Content-Type: application/json" -d
'{"description":"sin titulo"}'
HTTP/1.1 400 Bad Request

{"error":"Title is required"}

$ curl -i -X POST http://localhost:3001/tasks -H "Content-Type: application/json" -d
'{"title":"Test","status":"en_progreso"}'
HTTP/1.1 400 Bad Request

{"error":"Status must be one of: pending, completed"}
```

5. Informe de la prueba de persistencia

Esta sección responde al punto 10.4 del enunciado, que solicita describir qué ocurrió con los datos durante la prueba de persistencia y explicar por qué.

5.1 Procedimiento y resultado

Se creó una tarea nueva mediante `POST /tasks`, se reinició únicamente el contenedor de la API con `docker compose restart api` y se volvió a consultar el listado:

```
# 1. Crear la tarea de prueba
$ curl -X POST http://localhost:3001/tasks -H "Content-Type: application/json" -d
'{"title":"Prueba de persistencia","description":"Debe sobrevivir al restart"}'

{"id":4,"title":"Prueba de persistencia","status":"pending",
 "created_at":"2026-09-11T19:47:23.945Z","updated_at":"2026-09-11T19:47:23.945Z"}

# 2. Reiniciar SOLO el contenedor de la API
$ docker compose restart api
```

```
Container todo_api Restarting
Container todo_api Started

# 3. Consultar nuevamente tras el reinicio
$ curl http://localhost:3001/tasks

[{"id":2,"title":"Task 2","status":"pending","created_at":"2026-09-09T18:59:43.972Z"},
 {"id":3,"title":"Comprar leche y pan","status":"completed","updated_at":"2026-09-11T19:41:13.878Z"},
 {"id":4,"title":"Prueba de persistencia","status":"pending","created_at":"2026-09-11T19:47:23.945Z"}]
```

Resultado: las tres tareas siguieron presentes tras el reinicio, conservando exactamente los mismos `id` y las mismas marcas de tiempo previas. La tarea con `id` 1 no aparece porque había sido eliminada intencionalmente durante la prueba del endpoint DELETE.

5.2 Análisis: por qué ocurre esto

El resultado se explica por la separación de responsabilidades entre los dos contenedores. El contenedor `api` no almacena ningún dato: es un proceso sin estado que únicamente traduce peticiones HTTP en consultas SQL. Toda la información vive en el contenedor `db`, que a su vez escribe sus archivos en el volumen nombrado `todo_pgdata`, gestionado por Docker por fuera del ciclo de vida de los contenedores.

Al ejecutar `docker compose restart api` solo se detiene y vuelve a iniciar el proceso de Node. El contenedor de PostgreSQL nunca se detuvo y el volumen jamás se tocó, de modo que al reconectarse la API encuentra la base exactamente en el mismo estado.

El contraste con la Práctica 01 es directo: allí las tareas vivían en un arreglo de JavaScript dentro de la memoria del proceso, por lo que cualquier reinicio del servidor las borraba por completo. Aquí el estado se externalizó a un motor de base de datos con almacenamiento propio.

Caso en que sí se pierden los datos: el comando `docker compose down -v`, a diferencia de `restart` o de un `down` sin la bandera, elimina explícitamente los volúmenes declarados. Al borrarse `todo_pgdata` desaparece el directorio de datos de PostgreSQL, y en el siguiente arranque el motor lo reinicializa desde cero y vuelve a ejecutar `init.sql`. El resultado es una base con las dos tareas de ejemplo originales y sin ninguna de las creadas durante las pruebas.

Esto evidencia que la unidad de persistencia no es el contenedor sino el volumen: mientras el volumen exista, los datos sobreviven a cualquier reinicio o recreación de los contenedores.

6. Ejercicios bonus implementados

Se resolvieron las tres actividades opcionales de la sección 10.5 del enunciado.

6.1 Validación del campo status

Se define una única lista de valores permitidos, aplicada en los tres puntos donde el campo puede ingresar al sistema: el filtro del `GET`, el `POST` y el `PUT`. Cualquier otro valor obtiene un 400.

```
const VALID_STATUSES = ['pending', 'completed']

if (status && !VALID_STATUSES.includes(status)) {
  return res.status(400).json({ error: `Status must be one of: ${VALID_STATUSES.join(', ')}` })
}
```

6.2 Paginación en GET /tasks

Se agregaron los parámetros `limit` y `offset`, con un valor por defecto de 20 registros y un tope de 100 para evitar consultas excesivamente grandes. Ambos se envían como parámetros de la consulta, manteniendo el criterio de seguridad.

```
const limit = Math.min(parseInt(req.query.limit, 10) || 20, 100)
const offset = parseInt(req.query.offset, 10) || 0

await pool.query('SELECT * FROM tasks ORDER BY id LIMIT $1 OFFSET $2', [limit, offset])
```

6.3 Dockerfile multi-stage

El Dockerfile se reescribió en dos etapas. La primera resuelve las dependencias; la segunda parte de una imagen limpia y copia únicamente `node_modules` ya resuelto junto con el código fuente, de modo que la caché de npm y los artefactos intermedios de la instalación no quedan en la imagen final.

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev

FROM node:20-alpine
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
EXPOSE 3001
CMD ["node", "server.js"]
```

7. Verificación de los criterios de aceptación

Criterio (sección 11 del enunciado)	Evidencia	Estado
-------------------------------------	-----------	--------

<code>docker compose up --build</code> levanta ambos servicios sin errores	Figura 1	Cumple
Los cinco endpoints responden con los códigos correctos, incluyendo 404	Figuras 2 a 6 y sección 4.7	Cumple
Ninguna consulta concatena valores del usuario en el string SQL	Sección 3.3	Cumple
Los datos persisten ante <code>restart api</code> y se pierden solo con <code>down -v</code>	Sección 5	Cumple
El campo <code>updated_at</code> se actualiza al ejecutar un PUT	Figura 5	Cumple

7.1 Trazabilidad de requisitos funcionales

ID	Requisito	Estado
RF-01	Listar tareas	Verificado
RF-02	Obtener tarea por ID	Verificado
RF-03	Crear tarea	Verificado
RF-04	Actualizar tarea (PUT)	Implementado
RF-05	Eliminar tarea (DELETE)	Implementado
RF-06	Filtrado por estado	Verificado

8. Conclusiones

La práctica permitió comprobar de manera concreta la diferencia entre un estado alojado en la memoria del proceso y uno externalizado a un motor de base de datos. El punto central no es solamente que ahora exista PostgreSQL, sino que el estado dejó de pertenecer al contenedor de la aplicación: el servicio `api` se volvió descartable y puede reiniciarse, recrearse o escalarse sin consecuencias sobre los datos.

También quedó clara la distinción entre detener contenedores y eliminar volúmenes, algo que en la práctica determina si una operación es reversible o destructiva. En cuanto a la implementación, el patrón `RETURNING *` resultó particularmente útil: con una sola consulta se resuelve simultáneamente la operación, la detección de filas inexistentes para responder 404 y la obtención del registro resultante, evitando una consulta previa de verificación.